

UNIT I: Programming Fundamentals, Python Architecture, Tokens & Operators

25% Exam Marks

1. CPython Execution Pipeline

Architecture



- Hybrid Pipeline:** Source compiled to Bytecode; executed by Python Virtual Machine (PVM).
- Platform Independent:** .pyc runs on any OS with CPython installed.

2. Variable Reference Model

Pointer Concept



- Variables are **names/references**, not memory boxes.
- `id(x)`: Memory address. `type(x)`: Object class.
- Interning:** Integers `[-5, 256]` share cached references.

3. Mutability & Tokens

Core Rules

Category	Data Types
Immutable	int, float, bool, str, tuple, frozenset
Mutable	list, dict, set, bytearray

- Indentation:** Strict 4 spaces (PEP 8); no braces `{}`.
- Identifiers:** Cannot start with digit; case-sensitive.
- Why no ++/--:** Integers immutable; `++x` is parsed as unary `+(+x)`. Must use `x += 1`.

4. Complete 9-Tier Python Operator Precedence Matrix (Highest to Lowest Priority)

Exam Guarantee

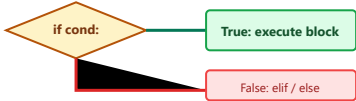
Rank • Operator	Associativity • Meaning	Rank • Operator	Associativity • Meaning	Rank • Operator	Associativity • Meaning
1. <code>()</code> , <code>[]</code> , <code>{}</code>	Left-to-right (Grouping, indexing, calls)	4. <code>*</code> , <code>/</code> , <code>//</code> , <code>%</code>	Left-to-right (Multiplication, division, modulo)	7. <code>&</code> → <code>^</code> → <code> </code>	Left-to-right (Bitwise AND, XOR, OR)
2. <code>**</code>	Right-to-left (Exponentiation: <code>2**3**2 == 512</code>)	5. <code>+</code> , <code>-</code>	Left-to-right (Addition, subtraction)	8. <code>==</code> , <code>!=</code> , <code><</code> , <code>is</code> , <code>in</code>	Left-to-right (Comparisons, identity, membership)
3. <code>+</code> , <code>-</code> , <code>~</code> , <code>x</code>	Right-to-left (Unary plus, minus, bitwise NOT)	6. <code><<</code> , <code>>></code>	Left-to-right (Bitwise left & right shift)	9. <code>not</code> → <code>and</code> → <code>or</code>	Short-circuit: <code>not</code> > <code>and</code> > <code>or</code>

UNIT II: Control Flow Branching, Loops & Function Modularity

25% Exam Marks

1. Conditional Decision Flow

Flowchart



Truthy vs Falsy: `0`, `0.0`, `""`, `[]`, `()`, `{}`, `None`, `False` are **Falsy**. All others are **Truthy**!

2. break vs continue vs pass

Loop Control

Primitive	Action	Loop else?
<code>break</code>	Exits entire loop immediately	Suppressed!
<code>continue</code>	Skips current cycle to next	Executes
<code>pass</code>	No-op (syntactic stub)	Executes
for-else: else runs ONLY if NO break!		

3. Function Arguments & LEGB

Modularity

- `*args`: Variable positional args → packed in **tuple**.
- `**kwargs`: Variable keyword args → packed in **dict**.
- LEGB Scope:** Local → Enclosing → Global → Built-in.
- `global x`: Mutates module-level variable.
- `nonlocal x`: Mutates enclosing closure variable.
- Lambda:** `sq = lambda x: x**2`

4. Date & Time Module (datetime) Cheat Card

Built-in Library

Function / Syntax	Description • Format Directives
<code>datetime.now()</code>	Returns current local system timestamp.
<code>dt.strftime("%d-%m-%Y")</code>	Object → String : <code>%d</code> (day), <code>%m</code> (month), <code>%Y</code> (4-digit year), <code>%H:%M:%S</code> (time).
<code>datetime.strptime(str, fmt)</code>	String → Object : Parses date string according to format specification.
<code>timedelta(days=7)</code>	Represents duration for arithmetic: <code>next_week = now + timedelta(days=7)</code> .

5. Top Exam Traps in Units 1 & 2

Avoid Negative Marking

- Mutable Default Trap:** `def f(x=[])` evaluates `x` once at definition! Always use `def f(x=None): if x is None: x = []`.
- Division Operators:** `7 / 2 == 3.5` (float), but `7 // 2 == 3` (floor quotient). `-7 // 2 == -4`!
- Short-circuit:** `[]` and `42` yields `[]`; `[1]` or `42` yields `[1]`.

UNIT III: Data Structures • Lists, Tuples & Dictionaries

25% Exam Marks

1. List Methods & Complexity Dynamic Array

Method	Cost	Operation
.append(x)	O(1)*	Appends item to array end
.pop()	O(1)	Removes & returns last item
.insert(0, x)	O(n)	Inserts at index; shifts items
.remove(val)	O(n)	Removes first match or ValueError
.sort()	O(n log n)	In-place Timsort algorithm

Slicing: seq[start : stop : step] • [::-1] reverses

2. Tuple Immutability & Rules Compact Struct

- **Single-item tuple:** t = (5,) • t = (5) is an int!
- **Packing / Unpacking:** a, b = 10, 20; head, *tail = [1,2,3].
- **Why Tuple:** Data integrity, lower memory footprint, faster iteration.
- **Can be Dict Key?** YES, if all tuple elements are themselves immutable!
- **Mutation Workaround:** t = tuple(list(t) + [new_val]).

3. Dictionaries & Key Properties Hash Map

- Key Requirements:
- 1. **Hashable & Immutable:** str, int, tuple. (list/dict cause TypeError: unhashable).
 - 2. **Keys are Unique:** Duplicate keys overwrite preceding value.
- d.get(k, default): Safe lookup; avoids KeyError.
 - d.pop(k): Removes key and returns its value.
 - {k: v for k, v in seq}: Dict Comprehension.

UNIT IV: File Handling & Object-Oriented Programming (OOP)

25% Exam Marks

1. Complete 9-Tier File Access Modes Cheat Matrix File Streams

Mode	Pointer Placement	File Exists Requirement	Truncation Effect
'r'	Beginning (offset 0)	File must exist (else FileNotFoundError)	No truncation.
'w'	Beginning (offset 0)	Created automatically if absent	Truncates immediately to 0 bytes!
'a'	End of File (EOF)	Created automatically if absent	No truncation; writes append to end.
'r+'	Beginning (offset 0)	File must exist	Overwrites data in-place from pointer.
'w+'	Beginning (offset 0)	Created automatically if absent	Truncates to 0 bytes!
'a+'	End of File (EOF)	Created automatically if absent	Appends writes; reads require seek(0).
'rb', 'wb'	Binary Stream equivalents	Same rules as above	Used for images, PDFs, compiled binaries.

with open("f.txt") as f: auto-closes via __enter__ / __exit__ • f.tell() & f.seek(pos, whence)

2. OOP Core, Dunder Methods & Operator Overloading Classes & Magic Methods

- **self:** Explicit first argument referencing the calling instance.
 - **Class vs Instance Vars:** Class vars declared in class body (shared); instance vars attached to self.var in __init__ (isolated).
- | Operation | Dunder Method | Operation | Dunder Method |
|-----------------------|---------------------|-------------|----------------------|
| Constructor | __init__(self, ...) | + Operator | __add__(self, other) |
| String Representation | __str__(self) | - Operator | __sub__(self, other) |
| Official / REPL Repr | __repr__(self) | == Operator | __eq__(self, other) |
| Destructor | __del__(self) | len(obj) | __len__(self) |
- Duck Typing:** "If it walks like a duck and quacks like a duck, it's a duck." Dynamic polymorphism based on behavior rather than explicit class type checking.

3. Multiple Inheritance & MRO (C3 Linearization) Diamond Resolution

```
graph TD; A[Class A] --> B["B(A)"]; A --> C["C(A)"]; B --> D["D(B, C)"]; C --> D
```

MRO: D → B → C → A → object

- Deterministic resolution via **C3 Linearization**.
- Inspect via ClassName.**__mro__** or **.mro()**.
- **super().method()** follows exact MRO order!

4. Composition vs Inheritance & Examiner Checklist Architecture

Inheritance ("Is-A")	Composition ("Has-A")
class Dog(Animal):	class Car: def __init__(self): self.engine = Engine()
Tight coupling; fragile base class risk.	Loose coupling; components easily mocked and swapped.

Topper Strategy: In university questions asking for operator overloading or classes, ALWAYS write **__init__**, **__str__**, and a test driver instantiation!